

Tutorial de HTML e CSS · do zero

Padoka Mágica

Construindo a landing page de uma padaria inspirada em *Puella Magi Madoka Magica*, passo a passo

Roteiro

1. Paleta de cores e temática
2. Estrutura do projeto
3. Cabeçalho
4. Rodapé
5. Hero
6. Nossa História
7. Como chegar
8. Contato
9. Responsividade
10. Desafios extras

Em cada etapa alternamos: primeiro o **HTML**, depois o **CSS**.
Projeto de referência: **Padoka_Magika**

ETAPA 1

Paleta de cores e temática

Antes de escrever código, vale decidir duas coisas: **quais cores** o site vai usar e **qual a personalidade** dele. Essas escolhas guiam todo o resto.

A paleta

São cinco cores, cada uma com um papel fixo no site, mais um branco levemente acinzentado para os detalhes neutros. Como a temática é *Madoka Magica*, cada tom lembra uma das garotas mágicas. No projeto, cada cor vira uma **variável CSS** (a coluna do meio).

Cor	Variável	Inspiração	Onde usamos
`#FFB6BB`	`--cor-rosa`	Madoka	Hover dos links, linha do rodapé, cards, fundo do Contato
`#FFE691`	`--cor-amarela`	Mami	Fundo do hero, logo e títulos do cabeçalho e do rodapé
`#95D5EE`	`--cor-azul`	Sayaka	Selo do hero, fundo do Como chegar, bordas do formulário
`#585872`	`--cor-roxa`	Homura	Cabeçalho, rodapé e cor do texto
`#9E4446`	`--cor-vermelho`	Kyoko	Títulos, botões e destaques
`#FAFAFA`	`--cor-branca`	neutra	Menu, etiquetas, bordas e caixas brancas

Por que variáveis?

Cada cor existe uma única vez, no arquivo `variables.css`. Se a padaria mudar de cor, você troca uma linha e o site inteiro acompanha. O texto também usa o roxo `#585872`, e não preto puro: contrasta bem com os fundos claros e combina com a paleta.

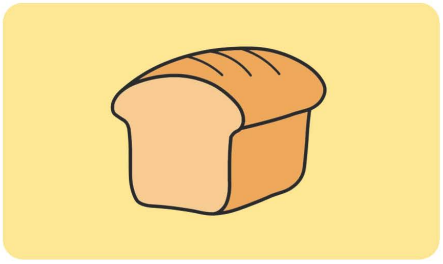
A temática

O nome **Padoka Mágica** é um trocadilho entre *padoca* e *Madoka Mágica*. As referências entram nos **textos** e nas **imagens**, sem mudar o layout:

Referência	Onde aparece
Mitakihara, a cidade do anime	Etiqueta do hero, história e endereço
Contratos e desejos (o gatinho branco)	Título do hero, história e formulário
As garotas mágicas	Nomes dos produtos e da equipe
Soul Gem, Grief Seed, Walpurgisnacht	Produtos, horário e rodapé
Loops temporais da Homura	Pão Integral Loop Temporal e um dos cards
A própria Madoka	Ilustração dentro do círculo do hero

Os arquivos de apoio

Antes de programar, separe o material que o site vai usar. Ele fica na pasta `assets`:



LogoImage.png
logo do cabeçalho e ícone da aba



heroImage.png
foto do pão, fundo do círculo do hero



Madoka.png
arte que fica por cima da foto

- **Fontes:** *Pacifico* (cursiva e divertida, para logo e títulos) e *Poppins* (limpa, para o texto corrido). Em vez de carregar do Google Fonts, os arquivos `.ttf` ficam dentro do projeto, em `assets/fonts`. Assim o site mostra as fontes certas mesmo sem internet.
- **Ícone:** `assets/icons/seta.svg`, uma seta para baixo que ainda não foi usada (vira um desafio no final).

Convenção de nomes

Para o código ficar previsível, o projeto segue duas regras:

- **Classes e ids em camelCase:** `heroTitulo`, `footerColuna`, `comoChegar`.
- **Variáveis CSS em minúsculas, com hífen e um prefixo que diz o tipo:** `--cor-roxa`, `--altura-header`.

A estrutura da página

Este é o mapa do que vamos construir, de cima para baixo:



ETAPA 2

Estrutura do projeto

Começamos pelo esqueleto: as pastas, o HTML vazio e as regras gerais do CSS. O CSS é dividido em **três arquivos**, cada um com uma responsabilidade.

2.1 · As pastas

Crie a pasta `Padoka_Magika` com esta organização (os arquivos de `assets` já estão prontos; o resto começa vazio):

```
Padoka_Magika/
├── index.html
├── css/
│   ├── variables.css   cores, alturas e fontes
│   ├── layout.css      o que se repete em toda página
│   └── home.css        o que é só da página inicial
├── js/
│   └── app.js          vazio por enquanto
└── assets/
    ├── fonts/          Pacifico e Poppins (.ttf)
    ├── icons/          seta.svg
    └── img/            LogoImage, heroImage e Madoka
```

- `variables.css` guarda os **valores** do site (cores, alturas, fontes).
- `layout.css` guarda o que se repete em **todas as páginas**: regras base, cabeçalho e rodapé.
- `home.css` guarda só o que pertence à **página inicial**: hero, história, como chegar e contato.

2.2 · HTML: o esqueleto

Onde colocar: todo o conteúdo inicial do `index.html`.

HTML index.html

```html
<!doctype html>
<html lang="pt-BR">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Padoka Mágica</title>
    <link rel="stylesheet" href="./css/home.css" />
    <link
      rel="shortcut icon"
      href="assets/img/LogoImage.png"
      type="image/png" />
  </head>

  <body>
  </body>
</html>
```

- `lang="pt-BR"` avisa que a página está em português, e o `charset UTF-8` garante acentos corretos.

- A `meta viewport` é essencial para o celular (voltamos a ela na etapa de responsividade).
- Só **um** arquivo CSS é ligado ao HTML: o `home.css`. Ele puxa os outros por `@import`, como veremos adiante.
- O segundo `link` define o ícone da aba do navegador, reaproveitando a imagem do logo.

2.3 · CSS: as variáveis

Primeiro arquivo de estilo: o que guarda os valores usados pelo resto.

Onde colocar: novo arquivo `css/variables.css`.

CSS css/variables.css

```css
:root {
  --cor-rosa: #ffb6bb;
  --cor-amarela: #ffe691;
  --cor-roxa: #585872;
  --cor-azul: #95d5ee;
  --cor-vermelho: #9e4446;

  --cor-branca: #fafafa;

  --altura-header: 7.5vh;
  --altura-footer: 15vh;
}

@font-face {
  font-family: "Fonte Titulo";
  src: url(../assets/fonts/Pacifico.ttf);
}

@font-face {
  font-family: "Fonte Corpo";
  src: url(../assets/fonts/Poppins-Regular.ttf);
  font-weight: normal;
  font-style: normal;
}

@font-face {
  font-family: "Fonte Corpo";
  src: url(../assets/fonts/Poppins-Bold.ttf);
  font-weight: bold;
  font-style: normal;
}

@font-face {
  font-family: "Fonte Corpo";
  src: url(../assets/fonts/Poppins-Italic.ttf);
  font-weight: normal;
  font-style: italic;
}

@font-face {
  font-family: "Fonte Corpo";
  src: url(../assets/fonts/Poppins-BoldItalic.ttf);
  font-weight: bold;
```

```
  font-style: italic;
}
```

- Tudo que está em `:root` vira uma **variável**: criamos com `--nome: valor` e usamos com `var(--nome)`.
- `vh` é uma unidade relativa à altura da tela: `7.5vh` é 7,5% dela.
- O `@font-face` registra uma fonte com um **nome que nós escolhemos** (`"Fonte Titulo"`, `"Fonte Corpo"`) e diz onde está o arquivo.
- A Poppins tem quatro arquivos, todos sob o mesmo nome. O `font-weight` e o `font-style` de cada um dizem ao navegador qual usar quando o texto estiver em negrito ou itálico.

> **Por que ../ no CSS e ./ no HTML?**
> Os caminhos são sempre relativos ao **arquivo onde estão escritos**. O `index.html` fica na raiz, então escreve `./assets/...`. O `variables.css` fica dentro de `css/`, então precisa subir um nível com `../` para chegar em `assets/`.

2.4 · HTML: as três caixas da página

Onde colocar: no lugar do `body` vazio. O conteúdo de cada caixa entra nas próximas etapas.

HTML index.html

```
<body>
  <header id="header" class="header"></header>

  <main id="content" class="content"></main>

  <footer id="footer" class="footer"></footer>
</body>
```

- Toda a página cabe em três blocos: **cabeçalho**, **conteúdo principal** e **rodapé**.
- O `id` serve de destino para links e para o JavaScript; a `class` serve para estilizar.

2.5 · CSS: a base e o encadeamento

Onde colocar: novo arquivo `css/layout.css`.

CSS css/layout.css

```
@import url(variables.css);

* {
  margin: 0px;
  padding: 0px;
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
  scroll-padding-top: var(--altura-header);
}
```

```css
body {
  font-family: "Fonte Corpo";
  color: var(--cor-roxa);
}

a {
  text-decoration: none;
}
```

Onde colocar: novo arquivo `css/home.css` (por enquanto só esta linha).

CSS `css/home.css`

```css
@import url(layout.css);
```



- O `@import` carrega outro arquivo CSS. Por isso o HTML liga só o `home.css`: ele importa o `layout.css`, que importa o `variables.css`.
- O `*` zera as margens e os espaços que o navegador coloca sozinho, e o `box-sizing: border-box` faz o `padding` e a borda entrarem **dentro** da largura do elemento.
- `scroll-behavior: smooth` faz a página deslizar ao clicar nos links do menu. O `scroll-padding-top` com a altura do cabeçalho faz o destino parar logo **abaixo** dele, e não escondido atrás.
- No `body`, usamos a fonte `"Fonte Corpo"` (Poppins) e a cor roxa, via `var()`.

A ordem dos arquivos importa

O `@import` precisa ficar no **topo** do arquivo. As regras do `home.css` vêm depois das do `layout.css`, e quando duas regras empatam, **vale a última**. Também vale saber que o navegador só descobre o `layout.css` depois de baixar o `home.css`; em sites grandes, costuma-se usar vários `<link>` no HTML. Para organizar um projeto de estudo em camadas, o `@import` funciona muito bem.

ETAPA 3

Cabeçalho

O cabeçalho tem o logo à esquerda e o menu à direita, e fica **fixo no topo** enquanto a pessoa rola a página.

3.1 · HTML do cabeçalho

Onde colocar: dentro do `<header>` que criamos na etapa 2 (substitua a tag vazia por este bloco).

HTML index.html

```html
<header id="header" class="header">
  <a href="./index.html#hero" class="logoContainer">
    <img src="./assets/img/LogoImage.png" alt="Logo da Padoka Mágica" />
    <h1>Padoka Mágica</h1>
  </a>

  <nav class="navigation">
    <a href="./index.html#hero">Início</a>
    <a href="./index.html#historia">Sobre</a>
    <a href="./index.html#comoChegar">Como chegar</a>
    <a href="./index.html#contato">Contato</a>
    <a href="">Cardápio</a>
  </nav>
</header>
```

- O logo é um `<a>` que envolve a imagem e o nome. Assim o conjunto todo é clicável e leva ao topo. O `alt` descreve a imagem para quem não pode vê-la.
- O menu fica dentro de `<nav>`, com um `<a>` por item.
- Cada `href` com `#` é um **link âncora**: leva ao elemento que tem aquele `id`. Por exemplo, `#historia` leva a `id="historia"`. Essas seções serão criadas nas próximas etapas.
- Os links começam com `./index.html` para que o mesmo cabeçalho funcione em outra página do site (como uma página de cardápio): o clique volta para a página inicial.
- O link **Cardápio** fica sem destino por enquanto. Ele vira um dos desafios no final.

3.2 · CSS do cabeçalho

Onde colocar: no `layout.css`, depois das regras base.

CSS css/layout.css

```css
.header {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  z-index: 100;

  background-color: var(--cor-roxa);
  color: var(--cor-amarela);

  height: var(--altura-header);
```

```
  padding: 15px 50px;

  display: flex;
  align-items: center;
  justify-content: space-between;
}

.logoContainer {
  display: flex;
  align-items: center;
  gap: 10px;
  color: inherit;
  transition: 150ms;
}

.logoContainer h1 {
  font-family: "Fonte Titulo";
  font-size: 2rem;
  font-weight: lighter;
}

.logoContainer img {
  height: calc(var(--altura-header) - 1vh);
}

.logoContainer:hover {
  scale: 102.5%;
}

.navigation {
  display: flex;
  gap: 30px;
}

.navigation a {
  color: var(--cor-branca);
  transition: 300ms;
}

.navigation a:hover {
  color: var(--cor-rosa);
}
```

- `position: fixed` com `top: 0` e `left: 0` prende o cabeçalho no topo da tela, e o `z-index: 100` o deixa **por cima** do resto do conteúdo.
- A altura vem da variável: `height: var(--altura-header)`. O `display: flex` com `justify-content: space-between` empurra logo e menu para as pontas, e o `align-items: center` centraliza na vertical.
- No `.logoContainer`, o `gap` separa imagem e nome, e o `color: inherit` faz o link usar o amarelo do cabeçalho em vez do azul padrão dos links.
- A Pacifico só existe no peso normal. Por isso o `h1` usa `font-weight: lighter`: sem isso o navegador "inventaria" um negrito e a letra ficaria deformada.
- A imagem tem `calc(var(--altura-header) - 1vh)`: o `calc()` faz contas misturando variável e unidade, deixando o logo 1vh menor que o cabeçalho.
- O `:hover` aumenta o logo 2,5% com a propriedade `scale`, e o `transition` suaviza a mudança.

Teste agora

Abra o `index.html` no navegador. O cabeçalho deve aparecer no topo, com o logo, o nome em amarelo e o menu em branco. Passe o mouse nos links: eles ficam rosa.

ETAPA 4

Rodapé

Com o cabeçalho pronto, fechamos a "moldura" da página com o rodapé. Assim, tudo que construirmos depois entra no meio, dentro do `<main>`.

4.1 · HTML do rodapé

Onde colocar: dentro do `<footer>` da etapa 2 (substitua a tag vazia por este bloco).

HTML index.html

```
<footer id="footer" class="footer">
  <div class="footerContainer">
    <div class="footerColuna">
      <p class="footerLogo">Padoka Mágica</p>
      <p>
        Padaria artesanal de Mitakihara desde 2011. Pão quentinho, chá da
        tarde e nenhum contrato escondido.
      </p>
    </div>
    <div class="footerColuna">
      <h3>Navegação</h3>
      <a href="./index.html#hero">Início</a>
      <a href="./index.html#historia">Sobre</a>
      <a href="./index.html#comoChegar">Como chegar</a>
      <a href="./index.html#contato">Contato</a>
      <a href="">Cardápio</a>
    </div>
    <div class="footerColuna">
      <h3>Fale com a gente</h3>
      <p>(11) 1234-5678</p>
      <p>(11) 91234-5678</p>
      <p>contato@padokamagica.br</p>
      <p>@padokamagica</p>
    </div>
  </div>
  <div class="copyright">
    <p>
      &copy; 2026 Padoka Mágica. Todos os direitos reservados. Nenhuma Soul
      Gem foi escurecida no preparo dos nossos produtos.
    </p>
  </div>
</footer>
```

- O `.footerContainer` guarda três colunas: apresentação, links de navegação e contatos.
- Os links do rodapé repetem os do menu, o que ajuda quem chegou ao fim da página.
- `©` é uma **entidade HTML** que vira o símbolo ©.

4.2 · CSS do rodapé

Onde colocar: no `layout.css`, depois do CSS do cabeçalho.

CSS `css/layout.css`

```css
.footer {
  padding: 50px 50px 20px;
  background-color: var(--cor-roxa);
  color: var(--cor-branca);
}

.footerContainer {
  display: flex;
  justify-content: space-between;
  gap: 30px;
  margin-bottom: 30px;
}

.footerColuna {
  display: flex;
  flex-direction: column;
  width: 30%;
}

.footerLogo {
  margin-bottom: 10px;
  font-family: "Fonte Titulo";
  color: var(--cor-amarela);
}

.footerColuna h3 {
  margin-bottom: 15px;
  color: var(--cor-amarela);
}

.footerColuna a {
  color: inherit;
  transition: 300ms;
  width: fit-content;
}

.footerColuna a:hover {
  color: var(--cor-rosa);
  transform: translateX(5px);
}

.copyright {
  border-top: 1px solid var(--cor-rosa);
  padding-top: 20px;
  text-align: center;
}
```

- O rodapé usa o mesmo roxo do cabeçalho: a página abre e fecha com a mesma cor.
- O `.footerContainer` é flex e coloca as três colunas lado a lado. Cada `.footerColuna` ocupa `30%` da largura.
- Dentro da coluna, `flex-direction: column` empilha os itens, um em cada linha.

- Os links têm `width: fit-content`: a área clicável fica só do tamanho do texto, em vez de ocupar a coluna toda. No `:hover`, o `translateX(5px)` dá um "empurrãozinho" para a direita.
- O `.copyright` tem uma linha rosa em cima (`border-top`) separando-o das colunas.

> **Atenção**
>
> O cabeçalho é `fixed`, então ele **não ocupa espaço** na página. Como o `<main>` ainda está vazio, parte do rodapé fica escondida atrás do cabeçalho. Isso se resolve a partir da próxima etapa, quando o conteúdo entrar no meio.

ETAPA 5

Hero

O **hero** é a primeira coisa que o visitante vê: um título grande, um texto curto e botões chamando para a ação. Vamos montá-lo em duas partes: primeiro o lado do texto, depois o lado da imagem.

5.1 · HTML do hero: o texto

Onde colocar: dentro de `<main>`.

HTML index.html

```html
<section id="hero" class="hero">
  <div class="heroTexto">
    <p class="heroTag">Padaria artesanal de Mitakihara</p>
    <h1 class="heroTitulo">Faça um contrato com o pão quentinho</h1>
    <p class="heroSubtitulo">
      Aqui ninguém precisa virar garota mágica pra ter um desejo
      realizado. Experimente o Croissant Rosa da Madoka, o Pão de Queijo
      da Charlotte e a Torta Tiro Finale da Mami, sempre com um chá bem
      quentinho.
    </p>
    <div class="heroBotoes">
      <a href="#contato" class="botao">Fazer meu pedido</a>
      <a href="#historia" class="botao botaoSecundario">
        Conheça a gente
      </a>
    </div>
  </div>
</section>
```

- O `id="hero"` é o destino do link **Início** e do logo, no cabeçalho.
- O segundo botão tem **duas classes**: `botao botaoSecundario`. Ele herda tudo de `.botao` e só muda o que for diferente.
- Os botões levam a `#contato` e `#historia`, seções que ainda vamos criar.

5.2 · CSS do hero: o texto

Onde colocar: no `home.css`, depois do `@import`.

CSS css/home.css

```css
.hero {
  display: flex;
  align-items: center;
  justify-content: space-between;
  min-height: 100vh;
  padding: 120px 50px 60px 50px;
  background-color: var(--cor-amarela);
}

.heroTexto {
  width: 50%;
}
```

```css
.heroTag {
  background-color: var(--cor-branca);
  color: var(--cor-vermelho);
  font-weight: bold;

  width: fit-content;

  padding: 6px 16px;
  margin-bottom: 20px;

  border-radius: 20px;
}

.heroTitulo {
  color: var(--cor-vermelho);

  font-family: "Fonte Titulo";
  font-size: 56px;
  font-weight: 300;
  line-height: 1.3;

  margin-bottom: 20px;
}

.heroSubtitulo {
  margin-bottom: 35px;
  font-size: large;
}

.botao {
  display: inline-block;
  background-color: var(--cor-vermelho);
  color: var(--cor-branca);

  border: 2px solid var(--cor-vermelho);
  border-radius: 30px;

  padding: 14px 30px;
  font-weight: bold;
  transition: 300ms;
}

.botaoSecundario {
  margin-left: 10px;
  background-color: transparent;
  color: var(--cor-vermelho);
}

.botao:hover {
  background-color: #742b2c;
  border-color: #742b2c;
  transform: translateY(-3px);
}

.botaoSecundario:hover {
  background-color: var(--cor-vermelho);
  border-color: var(--cor-vermelho);
```

```
  color: var(--cor-branca);
}
```

- `min-height: 100vh` faz o hero ter, no mínimo, a altura da tela inteira.
- O `padding` de `120px` em cima existe porque o cabeçalho fixo cobre o topo da página. Sem ele, o começo do hero ficaria escondido.
- A `.heroTag` usa `width: fit-content`: o fundo branco abraça só o texto, em vez de ocupar a linha toda.
- O `.botao` usa `display: inline-block` para ter largura e altura próprias e continuar na mesma linha do vizinho. No `:hover`, o `translateY(-3px)` levanta o botão.
- O `.botaoSecundario` tem fundo transparente: só a borda aparece, criando um botão "vazado".
- No botão secundário, as duas regras de `:hover` valem ao mesmo tempo e têm a mesma força. Quem vence é a **última** do arquivo, por isso `.botaoSecundario:hover` vem depois de `.botao:hover`.

5.3 · HTML do hero: a imagem

Onde colocar: dentro da `section` do hero, logo depois do `</div>` que fecha a `.heroTexto`.

HTML index.html

```
<div class="heroImagem">
  <div class="circulo">
    <img src="./assets/img/Madoka.png" alt="Madoka" class="madoka" />
    <img
      src="./assets/img/heroImage.png"
      alt="Pão Quentinho"
      class="pao" />
  </div>
  <p class="selo">Desde<br />2011</p>
</div>
```

Em vez de uma imagem só, o círculo tem **duas imagens empilhadas**: a foto do pão e a ilustração da Madoka por cima. O `<br />` quebra o texto do selo em duas linhas.

5.4 · CSS do hero: a imagem

Onde colocar: no `home.css`, logo depois do CSS dos botões.

CSS css/home.css

```
.heroImagem {
  width: 45%;
  display: flex;
  justify-content: center;
  position: relative;
}

.circulo {
  width: 380px;
  height: 380px;
  border: 10px solid var(--cor-branca);
  border-radius: 50%;
```

```css
  overflow: hidden;
  position: relative;
}

.madoka {
  position: absolute;
  width: 80%;
  bottom: 0;
  left: 10%;
  transform: translateY(-6.5%) scale(-0.75, 0.75);
  z-index: 2;
}

.pao {
  position: absolute;
  width: 100%;
  height: 100%;
  object-fit: cover;
  z-index: 1;
}

.selo {
  position: absolute;
  bottom: 20px;
  right: 60px;
  z-index: 3;

  height: 100px;
  width: 100px;

  background-color: var(--cor-azul);
  border: 5px solid var(--cor-branca);
  border-radius: 50%;

  display: flex;
  justify-content: center;
  align-items: center;
  text-align: center;
  font-weight: bold;

  transform: rotate(-12deg);
}
```



`z-index: 3`	.selo, o selo azul na frente de tudo
`z-index: 2`	.madoka, a arte com fundo transparente
`z-index: 1`	.pao, a foto preenchendo o círculo
`base`	.circulo recorta tudo em formato de círculo

- O `.circulo` é um quadrado (380 × 380) com `border-radius: 50%`, que o transforma em círculo. O `overflow: hidden` **recorta** tudo que passar da borda arredondada.

- As duas imagens usam `position: absolute`, então ficam uma sobre a outra, medidas a partir do círculo (que tem `position: relative`).
- A `.pao` tem `object-fit: cover`: a foto preenche o círculo, cortando as sobras em vez de esticar.
- A `.madoka` tem `width: 80%`, `bottom: 0` e `left: 10%`, o que a centraliza. O `scale(-0.75, 0.75)` reduz a imagem a 75%, e o valor **negativo** no primeiro número a **espelha** na horizontal.
- O `z-index` decide quem fica na frente: a Madoka (2) por cima da foto (1). Ele só funciona em elementos posicionados.
- O selo usa `position: absolute` em relação à `.heroImagem` e `rotate(-12deg)` para ficar inclinado, como um carimbo.

Por que o selo precisa de z-index?

A foto e a Madoka têm `z-index` positivo. Sem um valor no `.selo`, os dois passam por cima dele e cortam a letra "D" de "Desde". Com `z-index: 3`, o selo fica na frente de tudo.

ETAPA 6

Nossa História

Esta seção divide a tela em duas metades: à esquerda, o texto contando a história; à direita, quatro cards com números curiosos. No repositório ela existe só como uma `section` vazia, que vamos preencher agora.

6.1 · HTML da história: o texto

Onde colocar: dentro de `<main>`, logo depois do `</section>` do hero.

HTML index.html

```html
<section id="historia" class="historia">
  <h2 class="tituloSecao">Nossa História</h2>
  <div class="historiaConteudo">
    <div class="historiaTexto">
      <p>
        A Padoka Mágica nasceu em 2011, na cozinha do Seu Tomohisa, em
        Mitakihara. Enquanto a família saía para trabalhar e estudar, ele
        cuidava da horta e assava pães para os vizinhos, que logo
        começaram a fazer fila no portão toda manhã.
      </p>
      <p>
        Hoje a equipe cresceu. A Mami cuida das tortas e do chá da tarde,
        a Kyoko nunca deixa ninguém sair sem provar o Taiyaki de Maçã do
        Amor e a Sayaka recheia o famoso Sonho Azul Oceano. Já a Homura
        assa o Pão Integral Loop Temporal quantas vezes for preciso até
        ele sair perfeito.
      </p>
      <p>
        Nossa única regra: nenhum produto vem com contrato escondido. Se
        um gatinho branco aparecer no balcão oferecendo desejos, pode
        ignorar.
      </p>
    </div>
  </div>
</section>
```

- A classe `tituloSecao` vai se repetir em todas as próximas seções: escrevendo o estilo uma vez, ele vale para todos os títulos.
- A `.historiaConteudo` é a "caixa" que coloca texto e cards lado a lado.
- Esta seção é o destino do link **Sobre** do menu (`#historia`), e o botão "Conheça a gente" do hero também leva para cá.

6.2 · CSS da história: o texto

Onde colocar: no `home.css`, depois do CSS do hero.

CSS css/home.css

```css
.historia {
  padding: 100px 50px;
  background-color: var(--cor-branca);
```

```css
}

.tituloSecao {
  margin-bottom: 50px;

  font-family: "Fonte Titulo";
  font-size: 42px;
  font-weight: 300;
  text-align: center;
  color: var(--cor-vermelho);
}

.historiaConteudo {
  display: flex;
  align-items: center;
  gap: 50px;
}

.historiaTexto {
  width: 50%;
}

.historiaTexto p {
  margin-bottom: 20px;
  font-size: 17px;
  line-height: 1.8;
}
```

- No `.tituloSecao`, o `font-weight: 300` evita o negrito falso na Pacifico, como no título do hero.
- A `.historiaConteudo` é flex com `gap: 50px`, e a `.historiaTexto` ocupa metade da largura (`50%`).
- O `line-height: 1.8` aumenta o espaço entre as linhas e deixa o texto longo mais confortável de ler.

6.3 · HTML da história: os cards

Onde colocar: dentro da `.historiaConteudo`, logo depois do `</div>` que fecha a `.historiaTexto`.

HTML index.html

```html
<div class="historiaCards">
  <div class="card">
    <span class="cardNumero">2011</span>
    <p>Primeira fornada na cozinha do Seu Tomohisa</p>
  </div>
  <div class="card cardAzul">
    <span class="cardNumero">5</span>
    <p>Garotas mágicas trabalhando no forno e no balcão</p>
  </div>
  <div class="card cardAmarelo">
    <span class="cardNumero">40+</span>
    <p>Receitas, do Brigadeiro Grief Seed à Rosquinha Soul Gem</p>
  </div>
  <div class="card">
    <span class="cardNumero">∞</span>
    <p>Loops temporais da Homura até a fornada ficar perfeita</p>
```

```
  </div>
</div>
```

Todos os cards têm a classe `card`. Dois deles ganham uma segunda classe (`cardAzul` e `cardAmarelo`) para trocar a cor.

6.4 · CSS da história: os cards

Onde colocar: no `home.css`, logo depois do CSS do texto da história.

CSS `css/home.css`

```css
.historiaCards {
  width: 50%;
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  gap: 20px;
}

.card {
  width: 45%;
  padding: 25px;

  background-color: var(--cor-rosa);
  border-radius: 15px;
  text-align: center;
  transition: 300ms;
}

.card:hover {
  transform: scale(1.05);
}

.cardNumero {
  display: block;
  margin-bottom: 10px;

  font-size: 32px;
  font-weight: bold;
  color: var(--cor-vermelho);
}

.cardAzul {
  background-color: var(--cor-azul);
}

.cardAmarelo {
  background-color: var(--cor-amarela);
}
```

- `flex-wrap: wrap` deixa os cards "quebrarem" para a linha de baixo. Com `width: 45%`, cabem dois por linha.
- O `.cardNumero` é um `<span>`, que por padrão fica na mesma linha do texto. O `display: block` faz ele ocupar a linha inteira.

- O `.cardAzul` e o `.cardAmarelo` trocam o fundo rosa definido em `.card`. Como têm a mesma força, vale o que vem **depois** no arquivo: por isso ficam abaixo de `.card`.
- O `transform: scale(1.05)` no `:hover` aumenta o card em 5%.

ETAPA 7

Como chegar

Um mapa à esquerda e as informações práticas à direita: endereço, horário e transporte.

7.1 · HTML do Como chegar

Onde colocar: dentro de `<main>`, depois do `</section>` da história.

HTML index.html

```
<section id="comoChegar" class="comoChegar">
  <h2 class="tituloSecao">Como chegar</h2>
  <div class="comoChegarConteudo">
    <div class="mapa">
      <iframe
        src="https://maps.google.com/maps?
q=Pra%C3%A7a%20da%20S%C3%A9%2C%20S%C3%A3o%20Paulo&z=15&output=embed"
        loading="lazy"
        title="Mapa da Padoka Mágica"></iframe>
    </div>
    <div class="infoEndereco">
      <h3>Endereço</h3>
      <p>
        Rua Mitakihara, 123 - Centro<br />São Paulo - SP<br />CEP
        01000-000
      </p>

      <h3>Horário de funcionamento</h3>
      <p>
        Segunda a sábado: 6h às 20h<br />Domingo: 7h às 13h<br />Fechamos
        mais cedo em noite de Walpurgisnacht.
      </p>

      <h3>Transporte</h3>
      <p>
        A 5 minutos a pé da estação de metrô. Se no caminho você entrar no
        labirinto de uma bruxa, é só seguir o cheiro de pão quentinho.
      </p>
    </div>
  </div>
</section>
```

- O `<iframe>` mostra outra página dentro da nossa, neste caso o Google Maps.
- Para trocar o local, mude o texto depois de `q=` na URL. Outro caminho é abrir o endereço no Google Maps e usar **Compartilhar** → **Incorporar um mapa**, copiando o `src` do código gerado.
- `loading="lazy"` só carrega o mapa quando ele estiver perto de aparecer na tela, e o `title` descreve o iframe para leitores de tela.
- O `id="comoChegar"` é o destino do link **Como chegar** do menu.

7.2 · CSS do Como chegar

Onde colocar: no `home.css`, depois do CSS da história.

CSS `css/home.css`

```css
.comoChegar {
  padding: 100px 50px;
  background-color: var(--cor-azul);
}

.comoChegarConteudo {
  display: flex;
  gap: 40px;
}

.mapa {
  width: 60%;
}

.mapa iframe {
  display: block;
  width: 100%;
  height: 420px;

  border: 6px solid var(--cor-branca);
  border-radius: 15px;
}

.infoEndereco {
  width: 40%;
  padding: 35px;

  background-color: var(--cor-branca);
  border-radius: 15px;
}

.infoEndereco h3 {
  margin-bottom: 8px;
  font-size: 20px;
  color: var(--cor-vermelho);
}

.infoEndereco p {
  margin-bottom: 25px;
  line-height: 1.6;
}
```

- Dessa vez a divisão não é meio a meio: o mapa fica com `60%` e as informações com `40%`.
- O `display: block` no iframe tira um pequeno espaço que o navegador deixa embaixo dele por ele ser um elemento "de linha".
- A borda branca e o `border-radius` deixam o mapa com cara de "foto emoldurada".
- A caixa `.infoEndereco` é clara para se destacar do fundo azul.

Por que o mapa não aparece?

O mapa precisa de internet para carregar. Sem conexão, o espaço aparece vazio, só com a moldura branca.

ETAPA 8

Contato

A última seção é um formulário centralizado para encomendas e dúvidas. Também vamos montá-lo em duas partes: primeiro os campos de texto, depois as listas de opções, a mensagem e o botão.

8.1 · HTML do contato: os primeiros campos

Onde colocar: dentro de `<main>`, depois do `</section>` do Como chegar.

HTML index.html

```
<section id="contato" class="contato">
  <h2 class="tituloSecao">Contato</h2>
  <p class="contatoTexto">
    Quer fazer uma encomenda ou tirar uma dúvida? Manda uma mensagem!
    Prometemos realizar seu pedido sem cobrar sua alma.
  </p>

  <form id="formContato" class="formulario" action="#">
    <label for="nome">Nome</label>
    <input
      type="text"
      id="nome"
      name="nome"
      placeholder="Ex: Madoka Kaname"
      required />

    <label for="email">E-mail</label>
    <input
      type="email"
      id="email"
      name="email"
      placeholder="seuemail@mitakihara.com"
      required />

    <label for="telefone">Telefone</label>
    <input
      type="tel"
      id="telefone"
      name="telefone"
      placeholder="(11) 99999-9999" />
  </form>
</section>
```

- O `for` do `<label>` precisa ser igual ao `id` do campo. Assim, clicar no rótulo coloca o cursor no campo certo.
- Os tipos `email` e `tel` ajudam o navegador: o `email` confere se tem @, e no celular ambos abrem o teclado adequado.
- `required` impede o envio com o campo vazio, e o `placeholder` mostra um exemplo dentro do campo.
- O `id="contato"` é o destino do link **Contato** do menu e do botão "Fazer meu pedido" do hero.

8.2 · CSS do contato: o formulário

Onde colocar: no `home.css`, depois do CSS do Como chegar.

CSS `css/home.css`

```css
.contato {
  padding: 100px 50px;
  background-color: var(--cor-rosa);
}

.contato .tituloSecao {
  margin-bottom: 15px;
}

.contatoTexto {
  margin-bottom: 40px;
  font-size: 17px;
  text-align: center;
}

.formulario {
  display: flex;
  flex-direction: column;

  max-width: 600px;
  margin: 0 auto;
  padding: 40px;

  background-color: var(--cor-branca);
  border-radius: 20px;
  box-shadow: 0px 10px 30px rgba(158, 68, 70, 0.2);
}

.formulario label {
  margin-bottom: 6px;
  font-weight: bold;
}

.formulario input,
.formulario select,
.formulario textarea {
  margin-bottom: 20px;
  padding: 12px;

  border: 2px solid var(--cor-azul);
  border-radius: 8px;

  font-family: inherit;
  font-size: 15px;
  color: var(--cor-roxa);
}
```

- O `.contato .tituloSecao` (dois nomes separados por espaço) pega só o título **desta** seção e diminui a margem. Esse seletor é mais forte que `.tituloSecao` sozinho, então ganha mesmo que a ordem fosse outra.

- `max-width: 600px` com `margin: 0 auto` é a receita clássica para **centralizar** um bloco na horizontal.
- O `flex-direction: column` empilha rótulos e campos um embaixo do outro.
- Campos de formulário **não herdam** a fonte da página. O `font-family: inherit` manda herdar a Poppins do `body`.

8.3 · HTML do contato: opções, mensagem e botão

Onde colocar: dentro do `<form>`, logo depois do campo de telefone e antes de `</form>`.

HTML index.html

```html
<label for="assunto">Assunto</label>
<select id="assunto" name="assunto">
  <option value="encomenda">Encomenda</option>
  <option value="duvida">Dúvida</option>
  <option value="elogio">Elogio</option>
  <option value="desejo">Fazer um desejo (não recomendamos)</option>
</select>

<label for="produto">Produto</label>
<select id="produto" name="produto">
  <option value="croissant">Croissant Rosa da Madoka</option>
  <option value="paoQueijo">Pão de Queijo da Charlotte</option>
  <option value="torta">Torta Tiro Finale da Mami</option>
  <option value="taiyaki">Taiyaki de Maçã do Amor da Kyoko</option>
  <option value="sonho">Sonho Azul Oceano da Sayaka</option>
  <option value="integral">
    Pão Integral Loop Temporal da Homura
  </option>
  <option value="rosquinha">Rosquinha Soul Gem</option>
  <option value="brigadeiro">Brigadeiro Grief Seed</option>
  <option value="cafe">Café Walpurgisnacht (extra forte)</option>
</select>

<label for="mensagem">Mensagem</label>
<textarea
  id="mensagem"
  name="mensagem"
  placeholder="Escreva seu pedido aqui..."
  required></textarea>

<button type="submit" class="botao botaoEnviar">Enviar pedido</button>
```

- O `<select>` cria uma lista suspensa. O texto entre as tags `<option>` é o que a pessoa vê; o `value` é o que seria enviado.
- É no campo **Produto** que aparecem os nomes temáticos da Padoka Mágica.
- O botão reaproveita a classe `.botao` do hero e soma a `.botaoEnviar` para os ajustes.

8.4 · CSS do contato: foco, mensagem e botão

Onde colocar: no `home.css`, logo depois do CSS dos campos.

CSS `css/home.css`

```css
.formulario input:focus,
.formulario select:focus,
.formulario textarea:focus {
  outline: none;
  border-color: var(--cor-vermelho);
}

.formulario textarea {
  height: 130px;
  resize: vertical;
}

.botaoEnviar {
  margin-top: 10px;
  font-family: inherit;
  font-size: 16px;
  cursor: pointer;
}
```

- `:focus` é o estado do campo quando a pessoa clica nele. Trocamos o contorno padrão do navegador por uma borda vermelha.
- No `textarea`, o `resize: vertical` deixa a pessoa aumentar a caixa só para baixo, sem quebrar a largura do formulário.
- O `<button>` também não herda a fonte, por isso o `font-family: inherit`. O `cursor: pointer` mostra a "mãozinha" ao passar o mouse.

> **Importante**
>
> Nunca remova o `outline` do foco sem colocar outro destaque no lugar. Quem navega pelo teclado depende desse sinal para saber onde está.

ETAPA 9

Responsividade

O site está pronto no computador. Agora vamos adaptá-lo para telas pequenas. A ideia geral é simples: no celular, o que estava **lado a lado** passa a ficar **um embaixo do outro**.

9.1 · HTML: a meta viewport

Onde colocar: já está no `<head>` desde a etapa 2. Só confira se ela está lá.

HTML index.html

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

Sem essa linha, o celular finge ter uma tela de computador e mostra o site inteiro miniaturizado. Com ela, a largura da página passa a ser a largura real do aparelho, e as regras de responsividade conseguem funcionar.

9.2 · CSS: cabeçalho e rodapé

Onde colocar: no **final** do `layout.css`, depois de todas as outras regras.

CSS css/layout.css

```
@media (max-width: 768px) {
  html {
    scroll-padding-top: 170px;
  }

  .header {
    height: auto;
    padding: 12px 20px;
    flex-direction: column;
    gap: 10px;
  }

  .navigation {
    flex-wrap: wrap;
    justify-content: center;
    gap: 10px 20px;
  }

  .footer {
    padding: 40px 20px 20px;
  }

  .footerContainer {
    flex-direction: column;
  }

  .footerColuna {
    width: 100%;
  }
}
```

- `@media (max-width: 768px)` significa: "aplique estas regras só quando a tela tiver **até 768px** de largura". Esse valor é comum para separar tablets e celulares das telas maiores.
- A media query fica **no final** do arquivo porque, quando duas regras têm a mesma força, vale a que vem por último. Cada media query fica no arquivo das regras que ela altera.
- O menu agora quebra em duas linhas, então o cabeçalho precisa **crescer**: trocamos a altura fixa por `height: auto` e empilhamos logo e menu com `flex-direction: column`.
- O `flex-wrap: wrap` deixa os links passarem para a linha de baixo quando não couberem. No `gap: 10px 20px`, o primeiro valor é o espaço entre linhas e o segundo, entre colunas.
- Como o cabeçalho ficou mais alto, o `scroll-padding-top` sobe para `170px`, e os links âncora continuam parando abaixo dele.
- No rodapé, as colunas também se empilham e passam a ocupar `100%` da largura.

9.3 · CSS: hero, história, como chegar e contato

Onde colocar: no **final** do `home.css`, depois de todas as outras regras.

CSS `css/home.css`

```css
@media (max-width: 768px) {
  .hero {
    flex-direction: column;
    padding: 190px 20px 60px;
  }

  .heroTexto,
  .heroImagem {
    width: 100%;
  }

  .heroTexto {
    text-align: center;
  }

  .heroTag {
    margin: 0 auto 20px;
  }

  .heroTitulo {
    font-size: 38px;
  }

  .botaoSecundario {
    margin-left: 0;
    margin-top: 10px;
  }

  .circulo {
    width: 250px;
    height: 250px;
    margin-top: 40px;
  }

  .selo {
    width: 85px;
    height: 85px;
```

```
    right: 20px;
    font-size: 14px;
  }

  .historia,
  .comoChegar,
  .contato {
    padding: 70px 20px;
  }

  .tituloSecao {
    font-size: 34px;
  }

  .historiaConteudo,
  .comoChegarConteudo {
    flex-direction: column;
  }

  .historiaTexto,
  .historiaCards,
  .mapa,
  .infoEndereco,
  .card {
    width: 100%;
  }

  .formulario {
    padding: 25px;
  }
}
```

- Toda caixa flex que tinha itens lado a lado ganha `flex-direction: column`: o hero, a história e o Como chegar.
- As larguras em `%` (50%, 60%, 40%...) viram `100%`, para cada bloco ocupar a tela toda.
- Diminuímos fontes, o círculo do hero e os `padding` laterais (de `50px` para `20px`).
- O hero ganha `190px` de espaço em cima, porque o cabeçalho do celular é mais alto.
- A `.heroTag` ganha `margin: 0 auto 20px` para ficar centralizada: blocos com `fit-content` não obedecem ao `text-align`.

Cuidado com o text-align no hero

É tentador centralizar todo o hero com `text-align: center` na `.hero`. Mas a `.pao` é uma imagem absoluta **sem** `top` e `left`, então o navegador a posiciona onde ela ficaria no fluxo normal, e esse ponto muda com o alinhamento do texto. Resultado: a foto desliza para o meio do círculo e fica só a metade. Por isso o `text-align: center` vai somente na `.heroTexto`.

Como testar

No navegador, aperte **F12** para abrir as ferramentas de desenvolvedor e depois **Ctrl + Shift + M** para ativar o modo de dispositivo. Escolha um celular na lista ou arraste a largura da tela e veja o layout mudar em 768px.

ETAPA 10

Desafios extras

O site está completo, mas o projeto já deixa algumas pontas soltas, ótimas para praticar:

1. **Página do cardápio.** O menu já tem o link **Cardápio**, ainda sem destino. Crie `cardapio.html` e `css/cardapio.css` (que começa com `@import url(layout.css);`), reaproveitando o cabeçalho e o rodapé. Depois, preencha o `href` do link no menu e no rodapé.
2. **Variáveis que sobraram.** O `--altura-footer` está definido em `variables.css`, mas nenhuma regra usa. Use-o no rodapé ou apague-o. Aproveite para criar `--cor-vermelho-escuro` e trocar o `#742b2c` do `.botao:hover`, que está escrito direto no código.
3. **Menu hambúrguer.** No celular, esconda o menu e mostre um botão ☰ que o abre. A pasta `js` já tem o `app.js` vazio esperando por isso (lembre de ligá-lo com `<script src="./js/app.js"></script>` no fim do `body`).
4. **Seta de rolagem.** Use o `assets/icons/seta.svg` no fim do hero, como um convite para descer a página, apontando para `#historia`.
5. **Menos @import.** Troque os `@import` por três `<link>` no HTML (na ordem certa!) e compare o resultado.

Checklist final

- [ ] Os links do menu levam até as seções certas, parando abaixo do cabeçalho
- [ ] O cabeçalho continua no topo ao rolar a página
- [ ] O selo "Desde 2011" aparece inteiro na frente da foto
- [ ] Os botões e cards reagem ao passar o mouse
- [ ] Clicar em um rótulo do formulário coloca o cursor no campo
- [ ] O formulário não envia com campos obrigatórios vazios
- [ ] Em 768px ou menos, tudo fica empilhado e sem rolagem para os lados

Pronto! A Padoka Mágica está aberta, sem nenhum contrato escondido.